МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РЕСПУБЛИКИ КАЗАХСТАН
СЕВЕРО-КАЗАХСТАНСКИЙ УНИВЕРСИТЕТ ИМ. М. КОЗЫБАЕВА
ФАКУЛЬТЕТ ИНЖЕНЕРИИ И ЦИФРОВЫХ ТЕХНОЛОГИЙ
КАФЕДРА ИНФОРМАЦИОННО-КОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ ПО ООП
МОДУЛЬ 4

Тема «Портфолио веб-разработчика»

Выполнили: студенты группы ИСУ(kz-us)-24-1
Головин Д. П.
Дедов А. А.

Проверила: Астапенко Н. В.

Петропавловск, 2026

Разработка интерфейса приложения портфолио веб-разработчика

Введение

В работе разработано настольное приложение «Портфолио веб-разработчика» на Python и Tkinter. Оно хранит карточки веб-проектов, помогает находить нужные кейсы и готовит сведения для дальнейшей публикации. В этом модуле рассматриваются объектная модель и графический интерфейс. [1]

Задачи: описать предметную область классами; реализовать наследование и полиморфизм; создать главное окно и меню; обеспечить добавление, редактирование и удаление карточек; показать сведения о проекте и окно «О программе».

1 Предметная область и классы

Предметная область описана шестью классами. Developer хранит сведения об авторе и создаёт Portfolio. Portfolio управляет проектами и назначает им уникальные идентификаторы. Абстрактный Project содержит общие поля карточки: название, описание, дату, статус, технологии, роль автора, результат и признак избранного проекта. Technology описывает технологию и формирует строку её отображения. [4, 5]

LandingProject и WebAppProject наследуются от Project и добавляют собственные поля: число страниц у сайта либо ссылку на репозиторий и серверный стек у веб-приложения. Подклассы по-разному реализуют get_main_link(), а общее get_info() использует тип и стек объекта. Это позволяет интерфейсу работать с разными видами проектов через единый список Project.

Распределение ответственности классов

Таблица 1 раскрывает назначение каждого класса, показанного на UML-диаграмме. Она помогает проследить, где хранятся данные, кто управляет коллекцией и в каких местах проявляются наследование и полиморфизм.

Класс	Ответственность в приложении	Связь с объектной моделью
Developer	Хранит сведения об авторе портфолио и создаёт объект Portfolio.	Связывает профиль автора с управляемой им коллекцией проектов.
Portfolio	Содержит список Project, назначает ID и выполняет операции добавления, удаления и поиска.	Управляет набором объектов и обеспечивает единый интерфейс для разных видов проектов.
Project	Абстрактный базовый класс с общими полями карточки и методом get_info().	Абстракция: объявляет get_main_link(), реализацию которого задают подклассы.
LandingProject	Описывает сайт или лендинг и хранит характерное для него число страниц.	Наследует общие данные Project и переопределяет get_main_link() для ссылки на демо.
WebAppProject	Описывает веб-приложение, включая репозиторий и серверный стек.	Наследует Project; возвращает ссылку на демо, а при её отсутствии — репозиторий.
Technology	Хранит название, категорию и версию технологии; формирует строку get_info().	Карточка проекта содержит набор объектов Technology, а не одну неструктурированную строку.

Полиморфизм виден при вызове get_main_link() через ссылку на Project: вызывающий код не проверяет тип каждой записи, а получает результат от нужного подкласса. Общая часть карточки и поведения остаётся в Project; отличия сайта и веб-приложения описаны в их специализированных классах.

2 Интерфейс приложения

Главное окно включает строку поиска, фильтры типа и статуса, переключатель «Только избранные», сортируемую таблицу, действия с карточками и подробную панель выбранного проекта. Форма меняет набор полей в зависимости от типа проекта. В карточке отображаются технологии, роль, результат и ссылки. Меню «Файл» содержит загрузку, сохранение, экспорт и пункт «Выход»; команда «О программе» открывает сведения о приложении. [1, 2, 3]

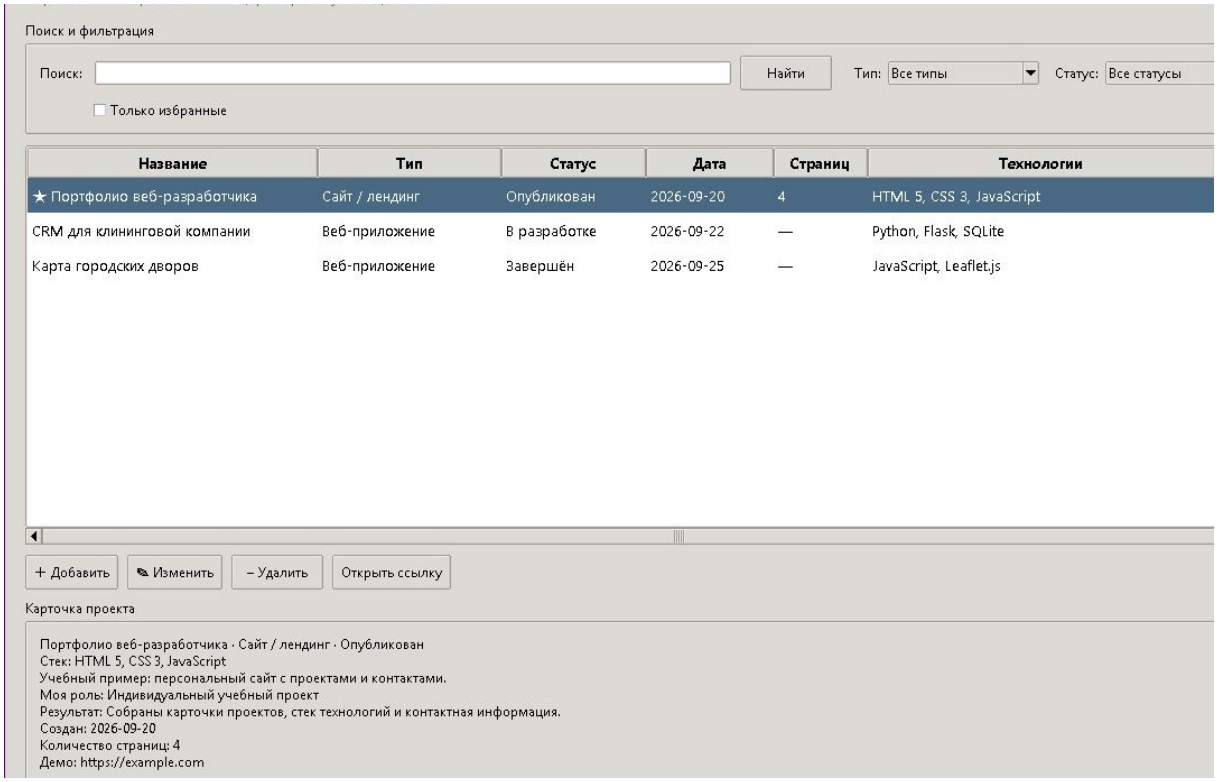


Рисунок 1 — Фрагмент главного окна: фильтры, проекты и карточка выбранного кейса

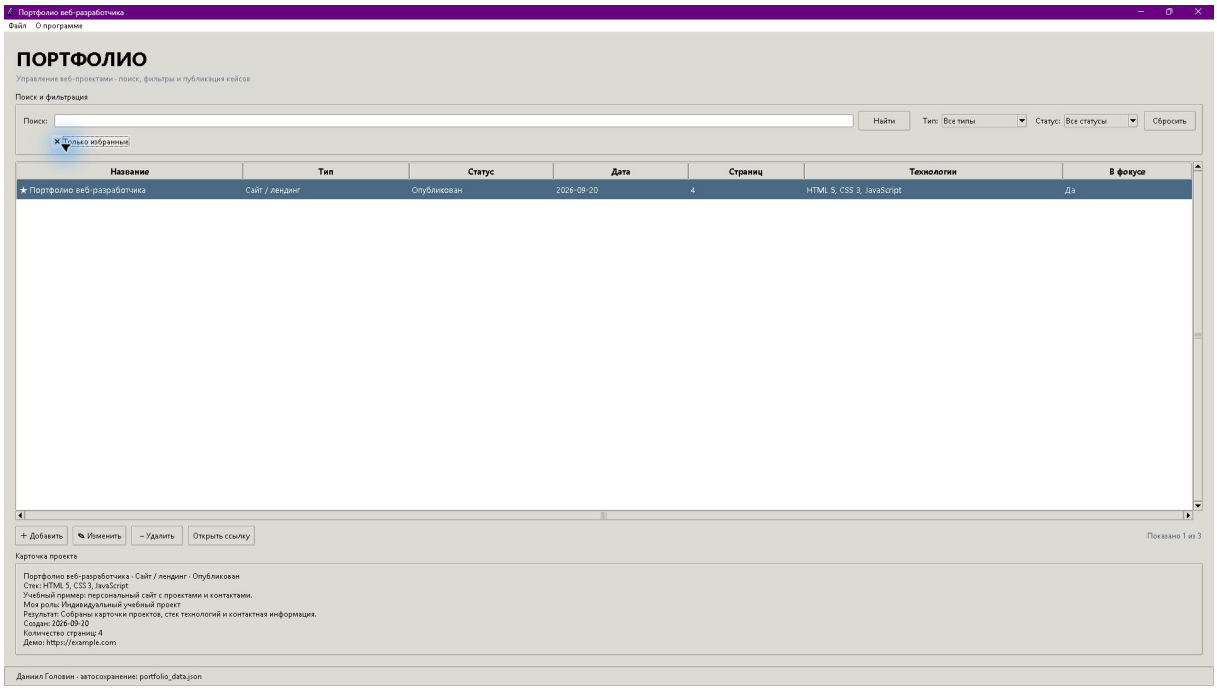


Рисунок 2 — Пример работы фильтра избранных проектов и карточки кейса

Фрагмент главного окна и действий с карточками

```
self.tree = ttk.Treeview(
    table,
    columns=("title", "type", "status", "date", "pages", "tech", "featured"),
    show="headings", selectmode="browse",
)
for label, callback in (("＋ Добавить", self.add), ("✎ Изменить", self.edit),
                        ("− Удалить", self.delete), ("Открыть ссылку", self.open_link)):
    ttk.Button(actions, text=label, command=callback).pack(side="left", padx=(0, 8))
selected_id = selected.project_id if selected else None
rows = self.portfolio.search(self.query.get())
self.visible = [p for p in rows if (kind == "Все типы" or p.project_type == kind)
                and (status == "Все статусы" or p.status == status)
                and (not self.featured_only.get() or p.featured)]
self._sort_visible()
self._render_visible(selected_id)
```

3 Основные алгоритмы

Добавление и редактирование: открыть форму → проверить обязательные поля → проверить дату, статус и ссылки → для сайта проверить положительное число страниц → создать объект нужного подкласса → добавить или заменить карточку → обновить список и автоматически сохранить JSON.

Удаление выполняется после выбора строки и подтверждения пользователя. Поиск без запроса возвращает все карточки; поиск с запросом проверяет основные поля, роль, результат, ссылки, число страниц и названия технологий. Выбранные тип и статус применяются как дополнительные фильтры; таблица сортируется по заголовкам, а избранные карточки можно показать отдельно.

Диаграммы и листинги программы

UML-диаграмма показывает шесть классов, наследование специализированных проектов, связь Portfolio с коллекцией проектов и состав карточки из объектов Technology.

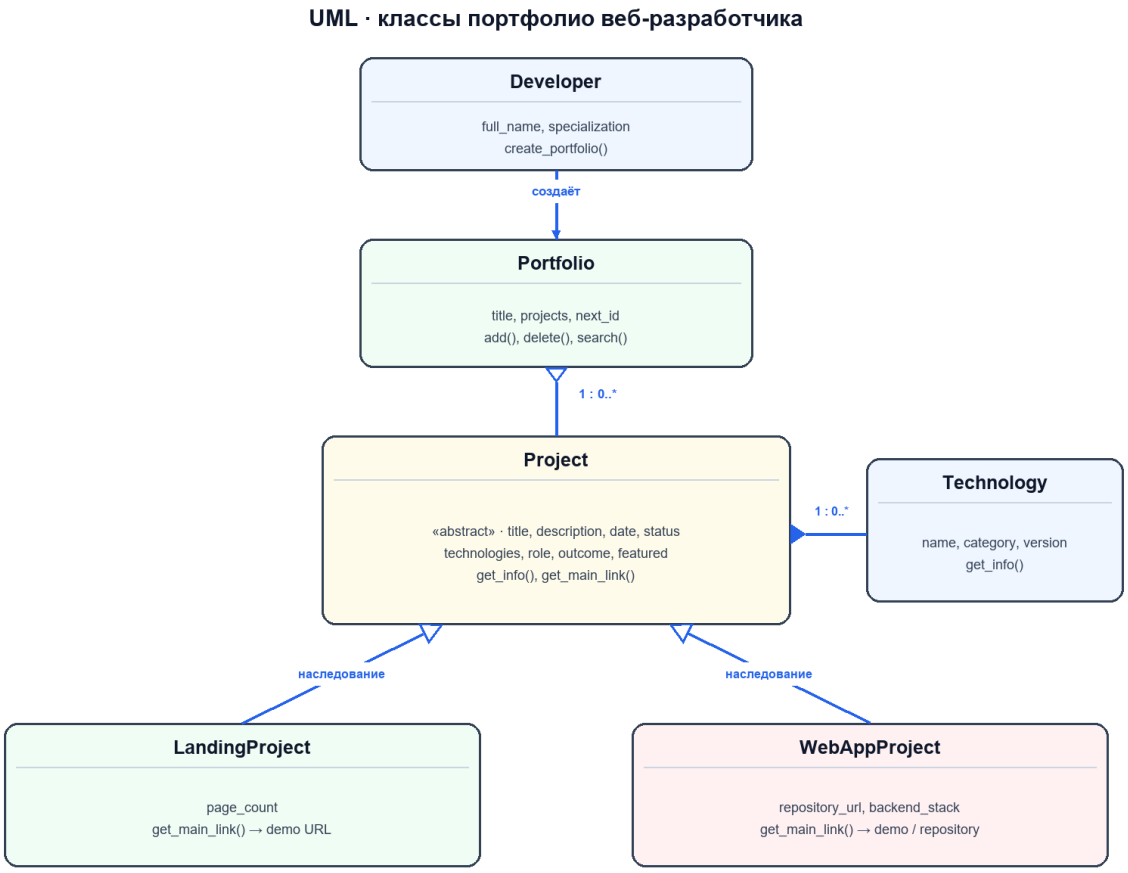


Рисунок 3 — UML-диаграмма классов приложения

Алгоритм добавления и редактирования

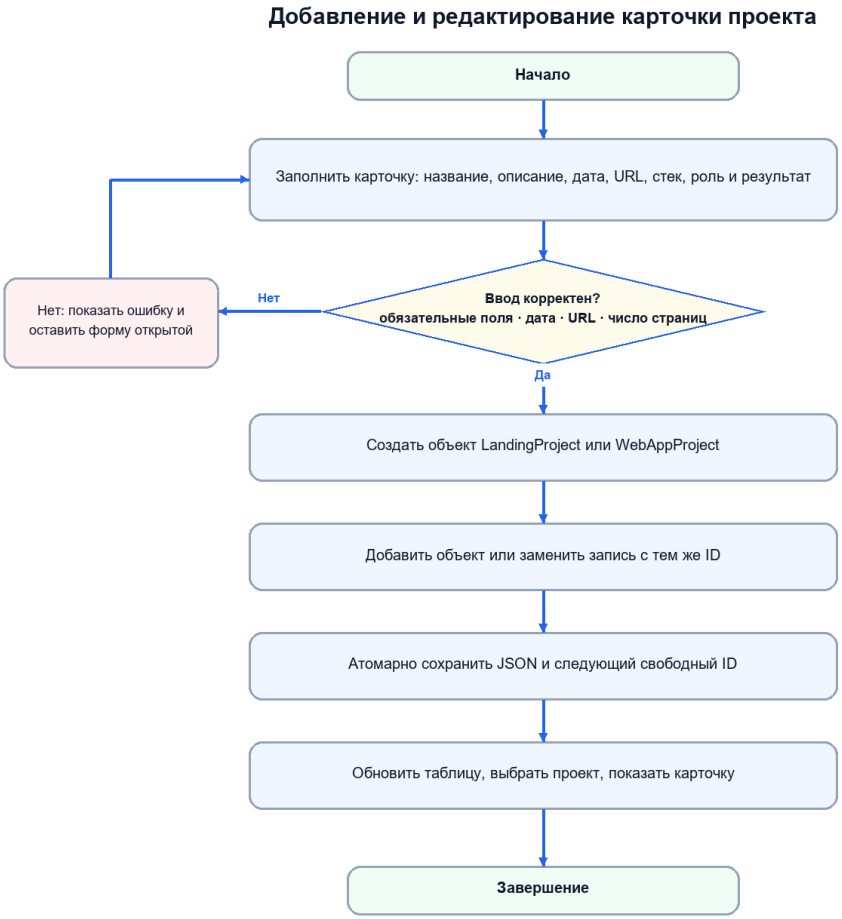


Рисунок 4 — Проверка формы и сохранение карточки

Алгоритм поиска и фильтрации



Рисунок 5 — Поиск и фильтрация проектов

Фрагмент реализации полиморфизма

```
class Project(ABC):
    @abstractmethod
    def get_main_link(self) -> str:
        ...

class LandingProject(Project):
    def get_main_link(self) -> str:
        return self.demo_url

class WebAppProject(Project):
    def get_main_link(self) -> str:
        return self.demo_url or self.repository_url
```

Форма ввода и фрагменты программы

В форме можно задать обязательные название и описание, дату, статус, ссылку на демо, технологии, личную роль и результат проекта. Для сайта доступны число страниц и отметка избранного; для веб-приложения — ссылка на репозиторий и серверный стек. При редактировании сохраняются идентификатор и метаданные уже выбранных технологий. Форма прокручивается: основные поля отделены от подробной истории кейса, метрик и настроек обложки. На рисунках 6–7 показаны начало формы и её нижняя часть с числом страниц, избранным и кадрированием обложки для компьютера и телефона.

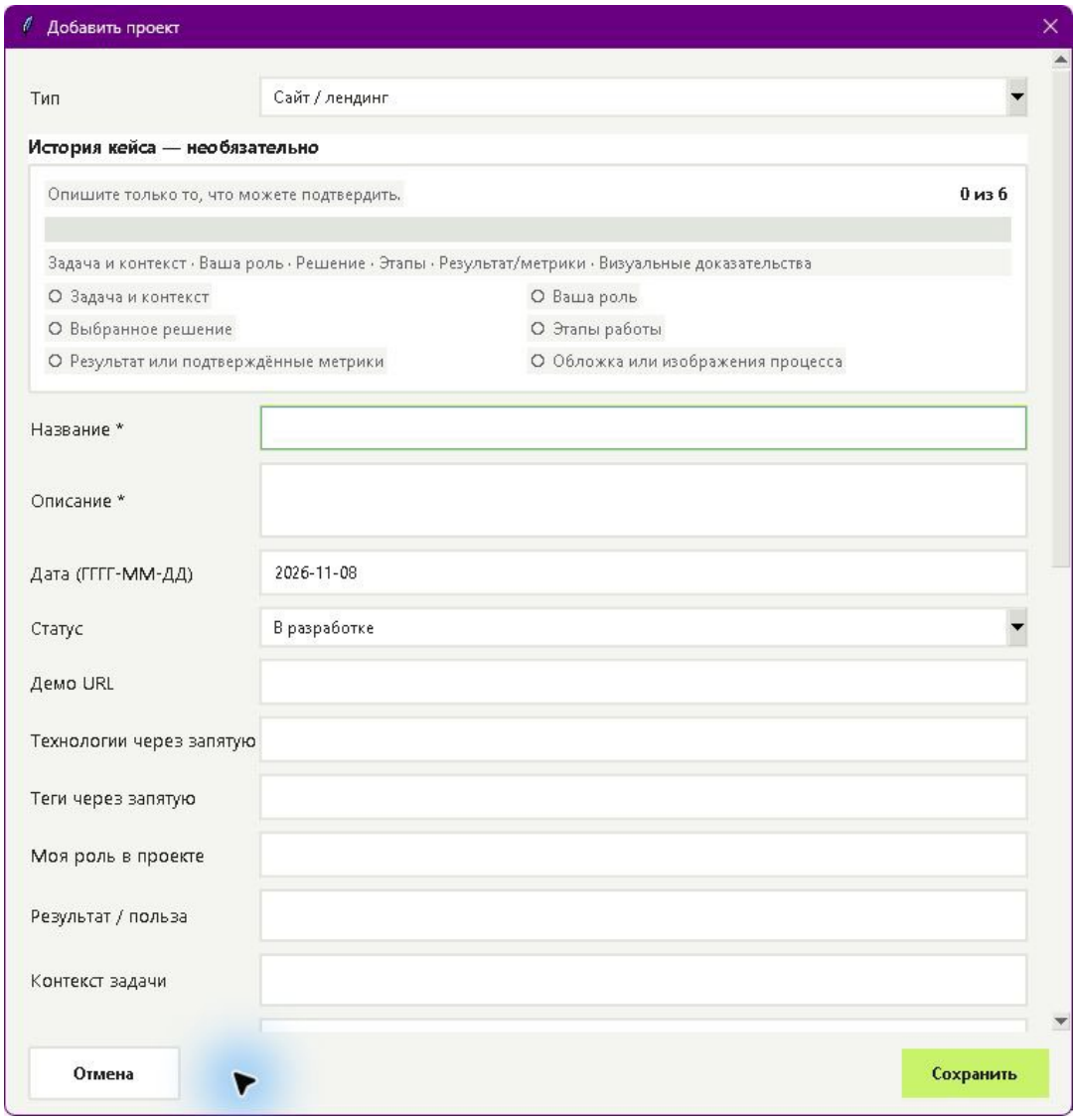


Рисунок 6 — Крупный снимок формы добавления проекта с обязательными полями и параметрами кейса

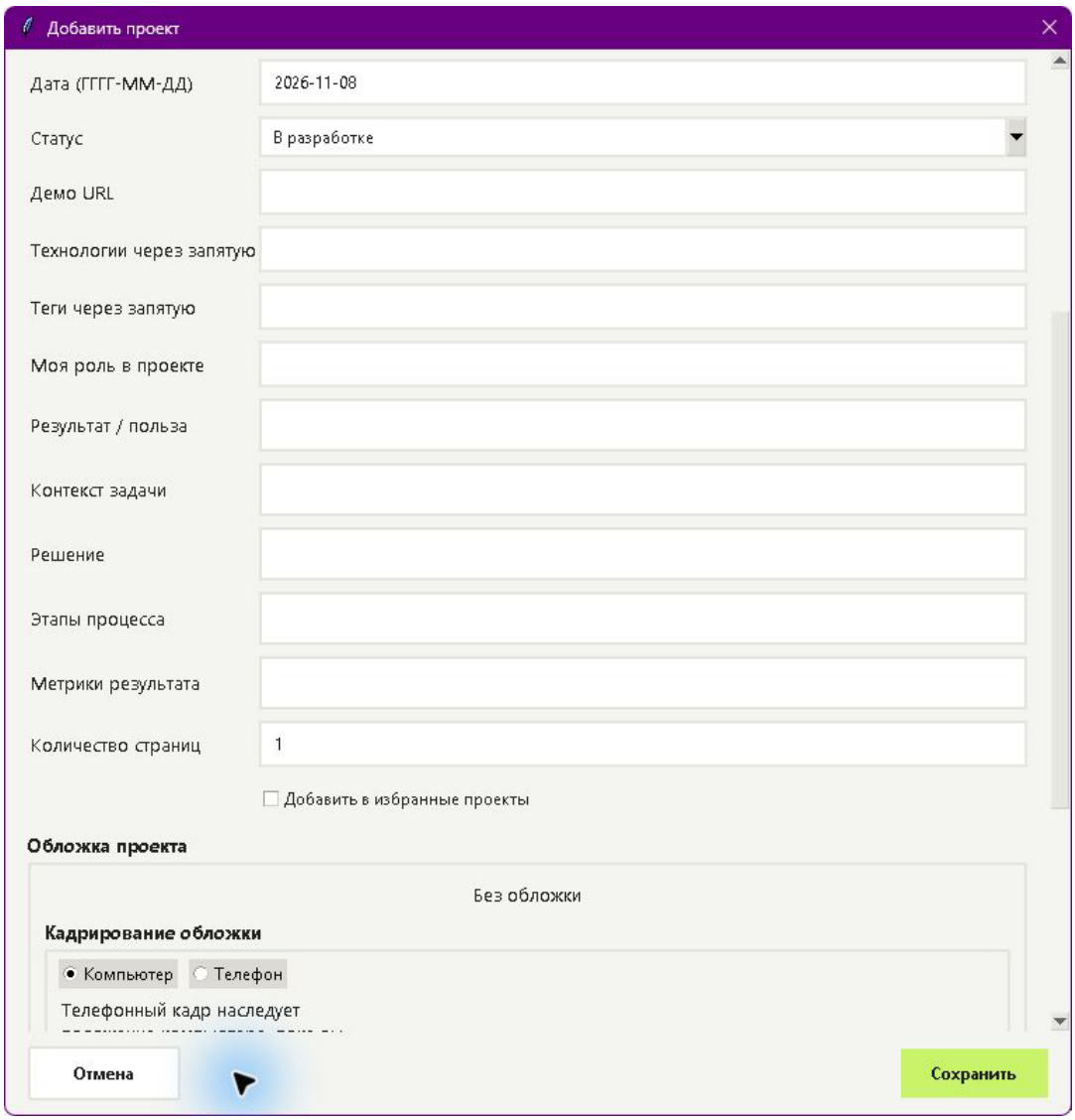


Рисунок 7 — Продолжение формы: метрики, число страниц, избранное и режим кадрирования обложки для компьютера и телефона

Фрагмент проверки данных

```
if not title.strip() or not description.strip():
    raise ValueError("Название и описание обязательны")
parsed_date = date.fromisoformat(created_at.strip())
if status not in PortfolioApp.STATUSES:
    raise ValueError("Выберите статус из списка")
parsed_url = urlparse(demo_url.strip())
if demo_url and (parsed_url.scheme not in ("http", "https") or not parsed_url.hostname):
    raise ValueError("Введите полную ссылку HTTP или HTTPS")
pages = int(page_count.strip())
if pages < 1:
    raise ValueError("Число страниц должно быть положительным")
```

Окно О программе

Команда меню открывает отдельное дочернее окно с кратким описанием продукта, графическим знаком на Canvas и кнопкой возврата к главной форме.

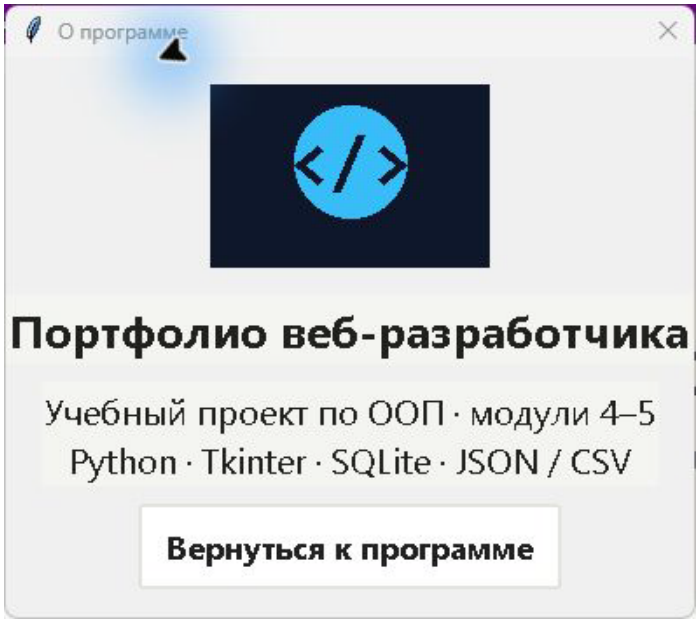


Рисунок 8 — Увеличенный снимок окна «О программе» с графическим знаком Canvas, описанием приложения и кнопкой возврата

Фрагмент создания окна «О программе»

```
win = tk.Toplevel(self.root)
win.title("О программе")
win.transient(self.root)
canvas = tk.Canvas(win, width=160, height=105, bg="#0f172a", highlightthickness=0)
canvas.pack(padx=20, pady=15)
canvas.create_oval(48, 12, 112, 76, fill="#38bdf8", outline="")
canvas.create_text(80, 44, text="</>", fill="#0f172a", font=("Consolas", 21, "bold"))
ttk.Label(win, text="Портфолио веб-разработчика", font=("Segoe UI", 13, "bold")).pack()
ttk.Label(win, text="Учебный проект по ООП · модули 4–5\nPython · Tkinter · JSON / CSV",
justify="center").pack(padx=20, pady=10)
ttk.Button(win, text="Вернуться к программе", command=win.destroy).pack(pady=(0, 16))
```

Заключение

Выполнены требования модуля: создано более четырёх классов, применены наследование и полиморфизм, реализованы меню, окно «О программе» с Canvas и операциями над карточками. Интерфейс дополнен динамическими полями формы, фильтром избранного, поиском по характеристикам и экспортом портфолио. Проект запускается локально и сохраняет изменения в JSON. Этот отчёт фиксирует этап модуля 4: настольная версия сохраняла данные в JSON. В модуле 5 основным локальным хранилищем стала SQLite, а JSON сохранился для миграции и обмена. Браузерная PWA-компаньон добавлена позднее как отдельный интерфейс.

Источники

1. Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html (дата обращения: 29.10.2026).
2. Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html (дата обращения: 29.10.2026).
3. Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html (дата обращения: 29.10.2026).
4. Python Software Foundation. abc — Abstract Base Classes. https://docs.python.org/3/library/abc.html (дата обращения: 29.10.2026).
5. Python Software Foundation. dataclasses — Data Classes. https://docs.python.org/3/library/dataclasses.html (дата обращения: 29.10.2026).